

PROVA-GT: PROvenance-centered Visual Analytics for Grounded Theory

Qifan Zhou , Qingqing Bi , Weihua Zhou , Duo Chen , and Siwei Fu 

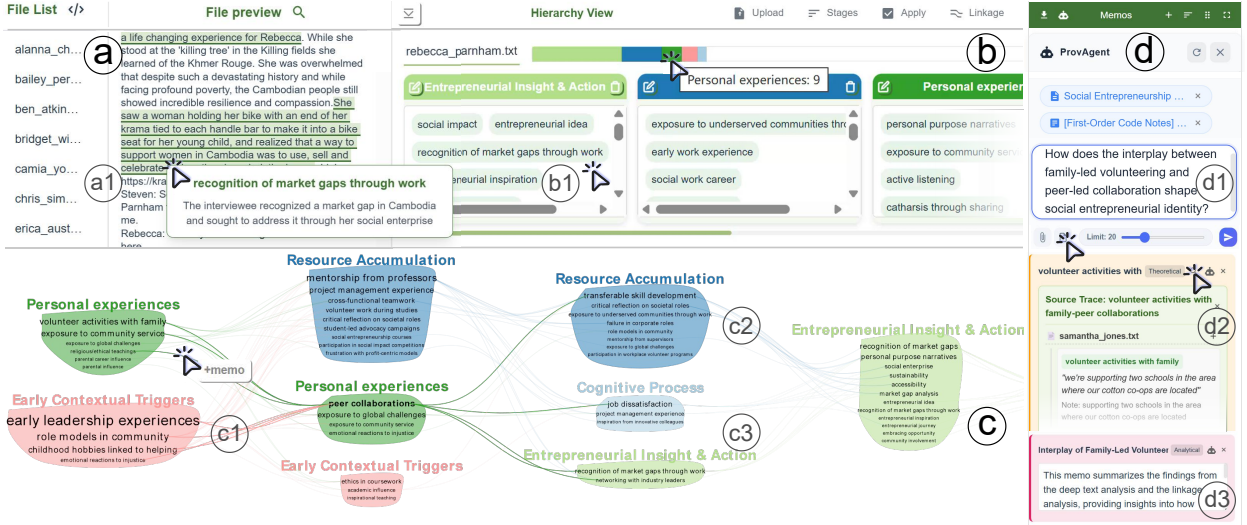


Figure 1: Applying PROVA-GT to analyze New Zealand-based social entrepreneurs' inspiration. (a) The *Content View* organizes textual data and supports open coding using prompt engineering. (b) The *Hierarchy View* supports categorization at both the file level and the corpus level. (c) The *Linkage View* visualizes complex relationships between hierarchical codes, assisting a user in axial coding and theorization. (d) The *ProvAgent* supports the backward process, enabling experts to trace the provenance of coding results and refine them in a unified interface.

Abstract—In social science research, grounded theory (GT) is widely adopted to analyze and code textual data. While large language models (LLMs) open new possibilities for coding at scale, they complicate the GT workflow by generating excessive codes and introducing hallucinations, posing challenges for experts to audit and revise the coding results. To address these challenges, we conducted a two-year design study and derived a series of domain needs and design tasks via interviews with scientists in entrepreneurial management. Subsequently, we presented PROVA-GT, a visual analytics system facilitating the GT workflow by supporting both the forward process of code generation and the backward process of validation and refinement, which are interconnected and iterative. Specifically, our system features a provenance workflow that leverages the agentic LLM and novel visual designs to assist experts in the backward process. To evaluate the usefulness of PROVA-GT and its generalizability in different domains, we reported two in-depth expert studies on entrepreneurial inspirations and trust erosion, respectively. We also conducted a user study to assess the effectiveness of the agentic workflow.

Index Terms—Grounded Theory, Agentic Workflow, Design Study

1 INTRODUCTION

In social science research, grounded theory (GT) [34] is a qualitative research method that emphasizes the flexible and iterative generation of new theories through inductive analysis, providing a deeper understanding of social phenomena [17, 33]. Coding is the core process of this method. It involves dissecting the data, forming concepts, and synthesizing them to gradually build a theoretical framework.

With the advent of large language models (LLMs), both academic [18, 63] and industrial [23] communities have been integrating LLMs into the coding process to address the labor-intensive limitations of manual coding [29, 57]. Though LLMs can generate numerous codes efficiently [38, 50], experts have to validate the coding results to ensure the reliability of the generated codes and the validity of the theories built upon them [22, 38]. Hence, they face three primary challenges when validating coding results assisted with LLMs.

First, the validation procedure is labor-intensive [69]. Despite clear prompts for guidance, LLMs struggle to propose reliable results due to limited domain knowledge [3, 6], the risk of hallucination [39], or bias [66]. To validate the coding results, experts need to frequently check the generated codes against the original data. The back-and-forth procedure is laborious and time-consuming [16]. Second, the amount of code that needs to be manually verified and managed is large in volume [54]. Since an LLM can effectively generate massive hierarchical codes for simple qualitative coding tasks [6], it poses a significant burden for experts in validation. Third, coding results are multi-level in nature [60]. As concepts are built and organized in a hierarchical

- Qifan Zhou, Weihua Zhou, Duo Chen, and Siwei Fu are with Zhejiang University, Hangzhou, China. E-mail: {zhou7fan, larryzhou, doraduochen, siwei.fu}@zju.edu.cn. Siwei Fu is the corresponding author.
- Qingqing Bi is with Nanyang Technological University, Jurong, Singapore. E-mail: qingqing.bi@ntu.edu.sg.
- Weihua Zhou and Siwei Fu are also with Zhejiang Key Laboratory of Decision Intelligence, Hangzhou, China.

Manuscript received xx xxx. 202x; accepted xx xxx. 202x. Date of Publication xx xxx. 202x; date of current version xx xxx. 202x. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org. Digital Object Identifier: xx.xxx/TVCG.202x.xxxxxxx

structure, verifying concepts at different levels and their relationships is critical yet challenging [16], requiring a comprehensive understanding of the data and the theoretical framework.

Visual analytics has proven to be effective in improving the qualitative coding process. For example, LLoM supports merging and splitting concepts [44] within a matrix view, while Semantic Concept Spaces [26] use Voronoi diagrams to partition hyper-conceptual regions, enhancing the structured display of high-level themes. Since these approaches focus on code generation, their effectiveness in code validation is questionable. Recently, some commercial software has explored the use of LLMs to query and reason over source data [5, 27]. Though helpful in fetching relevant information for validation, they do not support taking actions for efficient code refinement.

To address the above challenges, we conducted a two-year design study with scholars in social sciences to explore visualization designs and interactions for effective coding and validation. We derived a set of analytical tasks based on discussions and interviews with experts. Guided by these tasks, we developed PROVA-GT, an interactive system that supports the entire coding process of GT, including open coding, categorization, axial coding, and theorization. Since the GT workflow is iterative and flexible, PROVA-GT supports bi-directional interactions between source data and coding results (Figure 2). To be specific, PROVA-GT supports **the forward process** that generates hierarchical codes and relationships from source data, including first-order codes, second-order codes, and relationships between them. **The backward process**, on the other hand, is to validate the generated codes and relationships by tracing back to the original data (that is, provenance) and refining the coding results. Our system features ProvAgent, an agentic approach designed to facilitate provenance and iterative refinement. The core idea of ProvAgent is twofold. First, it provides a centralized interface to trace coding results back to their source data, assisting experts in validating the results based on their knowledge. Second, beyond validation, ProvAgent integrates an agentic workflow into refinement. Borrowing the idea of “vibe coding” [49], it predefines a set of tools that the LLM can use, including merging, splitting, creating, deleting, and modifying concepts and relationships, and employs the LLM in task planning and execution, allowing experts to efficiently and accurately revise the coding results.

To assess the effectiveness and utility of PROVA-GT in real-world scenarios, we designed case studies with five experts in two distinct fields, i.e., entrepreneurial management and marketing. Further, we conducted a lab study to evaluate the effectiveness of ProvAgent in assisting experts to validate the coding results. The evaluation demonstrates that PROVA-GT is effective in code generation and validation, facilitating data-driven theory building.

Our main contributions include: (1) A set of domain needs and design tasks derived through our two-year collaboration with domain experts. Guided by this, we designed PROVA-GT, an interactive system to support the bi-directional workflow of Grounded Theory. (2) We introduce ProvAgent, an agentic approach that supports the backward process of validating and refining coding results under expert confirmation. (3) Two case studies and a user study to evaluate the effectiveness and usefulness of PROVA-GT and ProvAgent with real-world scenarios.

2 RELATED WORK

2.1 Automated Approaches to Qualitative Analysis

Grounded theory remains a cornerstone of qualitative analysis [53]. While highly effective, it is also labor-intensive [29, 47]. Topic modeling [10] emerged as a promising technique to uncover hidden themes within textual data [7]. Despite its potential, the “bag-of-words” assumption lacks sensitivity to nuanced contextual meanings [40, 43], which can obscure critical semantic relationships within the text [20, 31]. To overcome these shortcomings, recent advancements have shifted toward embedding-based methods. For instance, BERTopic [35] transforms text into embedded representations, employing dimensionality reduction and clustering algorithms to derive topic representations. These automated methods, however, lack human intervention. As the

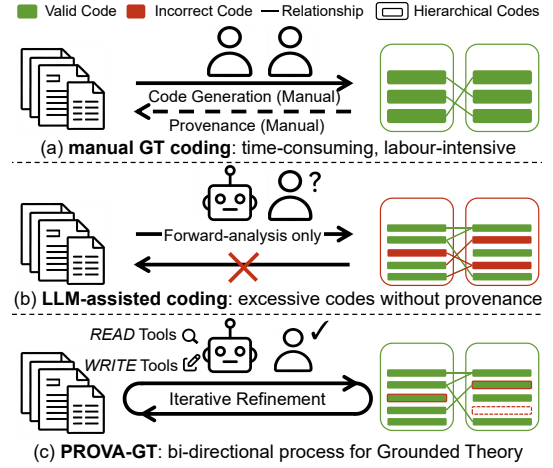


Figure 2: Illustration of three different coding workflows.

generated topics may not always align with the subtleties of human interpretation, experts typically seek to refine the automated results.

2.2 Human-in-the-loop Approaches

A “human-in-the-loop” approach requires researchers to continuously refine research hypotheses and frameworks throughout the coding process, with humans assuming a predominant role [28]. Systems such as Serendip [2] and Scholastic [37] have designed human-AI collaborative workflows aimed at facilitating qualitative coding at the word, document, and corpus levels. However, these early advancements primarily relied on statistical models and lacked depth in semantic understanding for automatic coding. Large language models (LLMs) have been applied to qualitative coding, helping experts build relationships between codes [13, 21, 58]. Recently, workflows such as CollabCoder [30] and AcademiaOS [59] have emerged, leveraging the capabilities of LLMs to construct relationships between codes. However, these workflows provide limited support for the backward process of auditing and refining coding results. Commercial software such as MaxQDA [42], NVivo [23], ATLAS.ti [5], and Hey Marvin [27] are designed for qualitative research and support both code generation and validation. Specifically, ATLAS.ti [5], MAXQDA [42], and Hey Marvin [27] employ LLMs to retrieve relevant information from source data and generate responses based on users’ queries. PROVA-GT shares many of the validation features provided by these tools. Differently, PROVA-GT moves one step forward by introducing a dedicated agentic workflow to automatically refine coding results under human confirmation. Our workflow reasons over evidence, generates action proposals, executes actions, and observes state changes, allowing users to refine coding results with minimal effort (Table 1). We also notice recent work in agentic LLMs, including LIDA [24], InsightLens [61], LightVA [67], and VizGenie [9]. Though inspiring, they are not tailored for the GT workflow, which is the focus of our work.

2.3 Provenance Visualization

Visualizing data provenance, or data lineage, is essential for understanding how data is derived, transformed, and updated over time [41, 52, 65]. Traditional approaches often represent data lineage graphs as node-link diagrams, such as VisTrails [15] and AVOCADO [56], to illustrate data pipelines. Other techniques, including Sankey diagrams [36], radial-based tree layouts [11], and Lament diagrams [4], have been applied to present schema-level data lineage. Recent advancements, such as the SOMNUS [64] and HYPNOS [14], utilize glyph-based designs and coordinated views to support interactive, instance-level lineage tracing for tabular data transformations. While these methods are effective for structured data, they are not designed to accommodate the semantic complexity in textual data. Some studies have supported provenance in textual analysis [19, 40, 45]. For instance, Semantic Concept Spaces partition a semantic space composed of word clouds to achieve classification, enabling direct

Table 1: Comparison of PROVA-GT with seven research projects and four commercial software. ✓: support; △: partially support; −: not support.

	► Forward Process		◄ Backward Process			
	Multi-level coding	Relationship mining	Manual validation	LLM-based reasoning	Action planning	Action execution
LLOOM [44]	✓	−	✓	−	−	−
HiCode [68]	✓	−	−	−	−	−
CollabCoder [30]	✓	−	✓	−	−	−
AcademiaOS [59]	✓	✓	−	−	−	−
Serendip [2]	△	−	✓	−	−	−
Scholastic [37]	✓	−	✓	−	−	−
SC Spaces [26]	✓	−	✓	−	−	−
ATLAS.ti [5]	✓	✓	✓	✓	−	−
MAXQDA [42]	✓	✓	✓	✓	−	−
NVivo [23]	✓	✓	✓	−	−	−
Hey Marvin [27]	✓	△	✓	✓	−	−
PROVA-GT	✓	✓	✓	✓	✓	✓

hierarchical adjustments through visual interactions [26]. Furthermore, systems like LLOOM [44] and HiCode [68] visualize LLM-generated codebooks, allowing a user to directly review text-to-code mappings and make manual adjustments. Although helpful, the aforementioned approaches are limited to code-level provenance and rely heavily on manual interactions during lineage tracing. Differently, PROVA-GT introduces an agentic approach for provenance. While supporting manual refinement for maximized flexibility, our system leverages the LLM for task planning, tool execution, and iterative refinement, enabling a user to efficiently trace the provenance of coding results and accurately refine them with minimal effort. To further clarify the difference, Table 1 compares PROVA-GT with representative research projects and commercial software in both the forward and backward processes of the qualitative coding workflow.

3 REQUIREMENTS ANALYSIS

The goal of this study is to design and develop an interactive system for facilitating qualitative analysis aligning with grounded theory (GT). Existing coding practice aligning with GT is diverse [32, 62]. Guided by a recent work on VIS4GT [25] and the discussion with our experts, we categorize the GT process into four main stages, i.e., open coding, categorization, axial coding, and theorization.

3.1 Working With Experts

The study involved a scholar (referred to as E1) and two of her graduate students (E2 and E3) from a business school, who actively participated in the entire research process. E1 specialized in entrepreneurial management and had recently collected an interview dataset from 358 New Zealand-based social entrepreneurs. Each interview lasted between 45 and 90 minutes, with an average transcript length of about 10,000 words. In qualitative research, this dataset proved to be too large to analyze manually, as the coding process required retaining substantial information to identify recurring ideas and concepts across the dataset. E1 and her group members had already utilized the LLM for generating first-order codes from a few interview transcripts and were satisfied with the results. However, they found that the LLM-generated second-order codes were unreliable, with E1 noting, “LLMs are nice. But they fall short in generating second-order codes. Probably because they lack a theoretical background of my research.” We held exploratory interviews and discussions with the three experts approximately every two weeks over a two-year-long collaboration. The entire study followed an iterative user-centered design process [55] and can be categorized into three stages as follows. In the first stage, we conducted formal interviews to identify existing problems and challenges in coding textual data. We had experts complete coding tasks using traditional methods, while observing the challenges they encountered when applying GT. Based on these interviews, we derived a set of analytical tasks to guide the development of PROVA-GT. For the second stage, we designed and implemented the prototype in an iterative manner, involving the three experts throughout the process. For demonstration purposes, we used the dataset provided by E1 to showcase how PROVA-GT performed on their dataset. Lastly, we

conducted two in-depth case studies with two groups of experts to assess the effectiveness and usefulness of PROVA-GT in real-world applications. The first case study involved analyzing entrepreneurial inspirations, aligning with E1’s primary research. While the second case study was conducted with two experts in marketing focusing on trust erosion in commercialization, it serves as a validation of PROVA-GT outside the specific domain of its initial development.

3.2 Understanding Domain Needs

N1. Unified coding system. One challenge faced by experts is the frequent switching between different tools during the coding process. For example, when generating first-order codes, they need to open Microsoft Word to show the original text, highlighting key text fragments and adding comments if necessary. Then, they switch to Microsoft Excel to organize the generated codes and build relationships between them. To accelerate the coding process, they would open an online LLM chatting interface using Chrome to gather the LLM-generated codes, and then copy and paste the results into Excel for further analysis. When checking the coding results, they need to switch back and forth between Word, Excel, and Chrome to verify the codes against the original text. E2 commented, “When I verify coding results across files. I have to open five or more (Word) files at the same time to see whether a code is reasonable.” Hence, a unified coding system that supports the iterative GT workflow is needed to reduce cognitive load and improve the experts’ efficiency.

N2. Human-centered coding. LLMs are helpful in open coding, where the codes are generally descriptive and do not require higher-order aggregations and interpretations. However, they pose new challenges in provenance. As E3 noted, “To generate reliable codes, I have asked LLMs to output raw text fragments as evidence for each code. But the text fragments may be faked. The more codes LLMs generate, the more hallucinations they may have.” Hence, experts seek to have full control during code generation. Besides, expert knowledge plays a critical role in categorization and axial coding, which depend not only on semantic alignment but also on the theoretical background of different research objectives. The process, however, is time-consuming, as it requires experts to review first-order codes in different files and assign them to a second-order code. Though substantial human involvement is necessary to ensure the accuracy and relevance of the thematic structure, experts seek intelligent approaches to reduce the burden.

N3. Building relationships. In axial coding and theorization, many kinds of relationships are usually discerned, e.g., causality, correlations, co-occurrence, etc. In this study, experts seek to identify frequent patterns as an initial process to uncover relationships, which lay the foundation for theorization. For example, experts want to understand if “early leadership experiences” and “social impact awareness” frequently appear consecutively in the narratives of social entrepreneurs. Similarly, they explored whether “frustration with existing systems” often occurs before “iterating based on user feedback” in the challenges faced by these entrepreneurs. To facilitate the exploration of frequent patterns, E1 proposed the concept of “Conceptual Stage”. For instance, when studying entrepreneurial motivation, a “Conceptual Stage” could represent an entrepreneur’s life stages, including “Childhood Stage,” “Education Stage,” followed by “Enterprise Stage.” By aligning first-order codes with a conceptual stage based on their context, experts can build frequent relationships among codes via stage order.

N4. Iterative coding provenance. The GT workflow is iterative, requiring auditing and revising the coding results based on new findings. As E1 noted, “No matter whether the results are human-generated or AI-generated, we need to validate and refine them round by round.” The provenance process, however, is tedious, as it requires switching between the source data and codes. For example, during open coding of an interview transcript, E2 initially labeled a fragment about a founder’s early life as “family financial struggle” based on a mention of economic hardship. As she processed more transcripts, she noticed recurring mentions of how such struggles motivated community involvement, prompting her to refine the code to “financial struggle as a trigger for social engagement” to better capture the emerging pattern. This requires

continuous iteration of coding standards to refine the first-order codes. Additionally, experts often need to revise the second-order codes. They refine code definitions when a second-order code cannot effectively generalize to first-order codes associated with it. Accelerating the provenance process is the key to an effective GT workflow.

3.3 Deriving Analytical Tasks

Based on the understanding of domain needs, we distilled five high-level analytical tasks (T1-T5).

T1. Support the entire GT procedure. To facilitate the iterative GT workflow (N1), our system should support all steps of GT from open coding to theorization. Besides the forward generation process, our system should provide an intuitive interface and interactions to allow iterative refinement of hierarchical codes and their relationships. Finally, after completing the GT procedure, the system should allow a user to download the results in a commonly used file format, e.g., CSV, for further application.

T2. Provide maximum customization. The system should provide maximum customization for the entire coding process (N2). This includes allowing a user to flexibly or manually revise first-order codes, define second-order codes, and classify first-order codes into second-order codes. This ensures that the thematic structure aligns not only with semantic meaning but also with the theoretical background and research objectives of the experts.

T3. Identify potential relationships between concepts. The system should allow experts to define conceptual stages, and establish relationships between codes based on the conceptual stage, thereby supporting experts in exploring frequent patterns (N3). As the patterns are large in volume, the system should automatically suggest relationships and display massive links in a readable manner.

T4. Support data-driven provenance. The system should provide sufficient details to support code provenance. That is, when presenting codes and relationships between them, the system should allow experts to view detailed information to validate the code or pattern (N4). The provenance process involves two steps. First, experts need to locate the text fragments associated with the codes and relationships. Second, they need to revise the coding results based on their knowledge and the evidence from text fragments.

T5. Ensure the reliability of LLM outputs. The system should guide the LLM to generate high-quality results and alleviate hallucinations (N2, N4). To achieve this, the system should make sure to 1) guide the LLM to retrieve enough reliable context for reasoning, 2) ask for user review and confirmation before taking actions, and 3) rely on pre-defined tools to perform specific actions.

4 PROVA-GT DESIGN

4.1 PROVA-GT Overview

Guided by design tasks, we have built a visual analytics system facilitating the iterative process of the GT workflow (T1), as shown in Figure 3. In the forward process (denoted as ►), the Content View presents raw text pieces and supports automatic **Open Coding** using prompt engineering. All first-order codes are shown in the Hierarchy View, ready for **Categorization**. After a user creates second-order codes, the Hierarchy View automatically builds a hierarchical coding structure by classifying first-order codes into second-order codes. To support **axial coding**, the Linkage View builds and visualizes relationships between hierarchical codes, and it supports **theorization** through aggregation and filtering techniques that allow a user to identify core patterns for theory building.

In the backward process (◄), PROVA-GT features **ProvAgent**, an agentic workflow that supports intelligent provenance and refinement. Specifically, after receiving a user query, ProvAgent starts planning and reasoning with the help of the *READ* tool. Then, it executes the action list using a set of pre-defined *WRITE* tools for reliability, including merging, splitting, creating, deleting, and modifying concepts and relationships. The process is iterative. In addition, to ensure flexibility and customization (T2), it builds bidirectional connections between all other views to support manual refinement and verification.

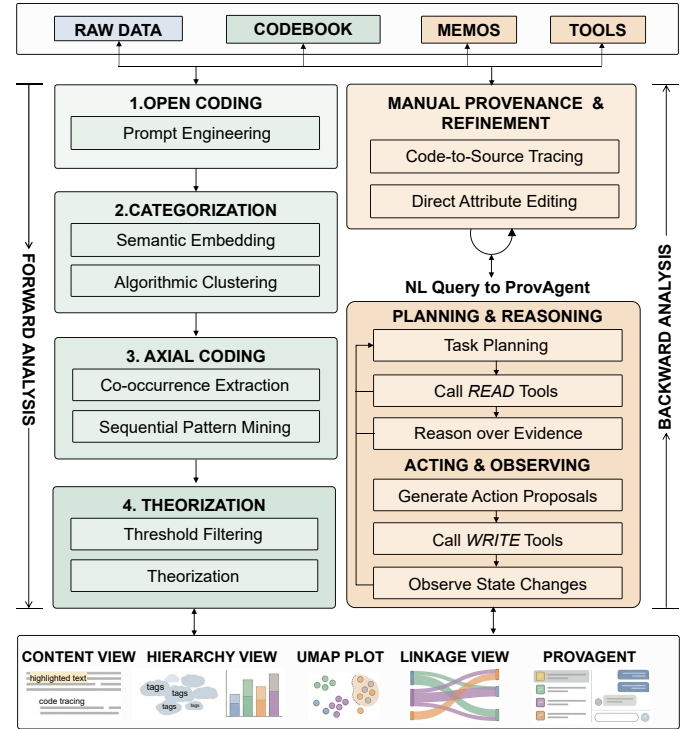


Figure 3: The overall architecture of PROVA-GT. It supports both the forward process of code generation and the backward process of provenance and refinement, which are interconnected and iterative.

4.2 Content View: Generating First-order Codes

► The Content View (Figure 1(a)) provides an intuitive entry point for open coding. Initially, it shows a file list and the content of a selected file. To leverage the LLM’s capabilities for open coding, the Content View offers comprehensive guidance on prompt engineering, including introduction, analysis objectives, and requirements for coding results. While surfacing prompts ensures flexibility, it poses a risk of accidental misuse. Therefore, PROVA-GT establishes a review mechanism to ensure security, as shown in the supplementary material (S1). The Content View sequentially processes the file list by passing the file content and prompts to an LLM (i.e., DeepSeek-Chat in our implementation) to generate first-order codes. The system maintains a codebook where each code includes a keyword, its explanation, the original sentence, and its conceptual stage. Then, the Content View highlights text fragments corresponding to first-order codes in green (Figure 1(a1)). Besides automatic generation, the Content View also supports manual coding, allowing a user to select text fragments and create first-order codes by filling in the code name and explanation.

◄ This view supports provenance and refinement with two operations. First, to revise a single first-order code, a user can double-click a text fragment to enter edit mode and modify the name and explanation of that code. Second, to systematically verify the code across files, a user can right-click a text fragment and click the “+memo” button to add it to ProvAgent (Section 4.5).

4.3 Hierarchy View: Categorization from Two Granularities

► The Hierarchy View (Figure 1(b)) is designed to support categorization at two levels of granularity: the file level and the corpus level. At the file level, this view presents all first-order codes using tag clouds, while second-order codes are shown as cards. After a user creates a card (i.e., a second-order code, Figure 4(a), (b)) with a name and explanation, the user can click the “absorb” button (Figure 4(d)) to invoke a semantic comparison between the second-order code and first-order codes. We set the cosine similarity threshold heuristically to 0.3, assigning each first-order code to the second-order code with the highest similarity only if that similarity exceeds 0.3. For example, after creating a card named “educational background” and clicking the “absorb” button, the system

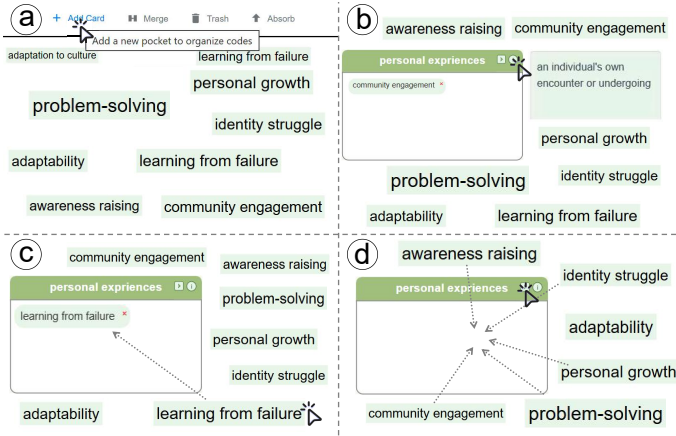


Figure 4: Categorization at the file level of the Hierarchy View. (a) Add a new card to organize codes. (b) Modify the name and description of the card. (c) Manually drag related first-order codes into a card. (d) Automatically group first-order codes after clicking the “absorb” button.

then evaluates the semantic similarity between all first-order codes and “educational background,” grouping phrases like “preschool education” and “university learning” under that card. To support the iterative process, a user can revise card explanations or manually fine-tune the coding results, ensuring the process remains user-led (Figure 4(c)).

► At the corpus level, this View is designed to extend user-generated cards across the entire corpus. Hence, PROVA-GT provides an “apply to all” button that allows a user to apply the existing hierarchical coding scheme from one file to the entire corpus (as shown in the supplementary material (S2)). To help experts assess the progress of categorization, we introduced stacked bars in which colored bars represent second-order codes, and their width corresponds to the number of first-order codes (Figure 5(a)). When a user clicks the “stack” button (Figure 5(a1)), all cards for each file are hidden while leaving the bars aligned compactly, which forms a stacked bar chart, as (Figure 5(b)(c)) shows. This provides a user with an overview of the coding results, enabling them to visually evaluate the effectiveness of categorization from quantitative perspectives.

◀ The Hierarchy View supports provenance and refinement at both the file level and the corpus level. At the file level, presenting code origins is crucial for assessing code quality. Hence, when a user clicks on a tag in Figure 1(b1), the system locates the corresponding file and highlights the text fragment in the Content View, helping the user trace the source and meaning of the code. To verify a second-order code at the corpus level, a user can right-click on a card and add the second-order code to ProvAgent (Section 4.5).

4.4 Linkage View: Supporting Data-Driven Theory Building

► The Linkage View is designed to help analysts explore frequent patterns between hierarchical codes (T3). First, a user is required to define conceptual stages when drafting prompts. For example, in the study of entrepreneurial motivation, “Conceptual Stage” can represent an entrepreneur’s life stages, including “Childhood Stage,” “Education Stage,” and “Enterprise Stage.” During open coding, the LLM automatically assigns first-order codes to these stages based on their context. For instance, “parental influence” and “childhood hobbies” are categorized under the “Childhood Stage.” Next, the system uses a Sequential Pattern Mining algorithm [1] to establish connections between codes. It then aggregates the same instances across all articles, increasing the weight of the connections between these first-order and second-order codes. This approach is similar to how sociologists gather evidence from multiple sources. Finally, the Linkage View employs a Sankey-based diagram to present the frequent patterns. Each column represents a conceptual stage, and a user can right-click on a first-order code to manually change the conceptual stage. The Linkage View provides a path-threshold slider. Links with a frequency below the threshold are excluded from the diagram, allowing users to focus on the most prominent connections.

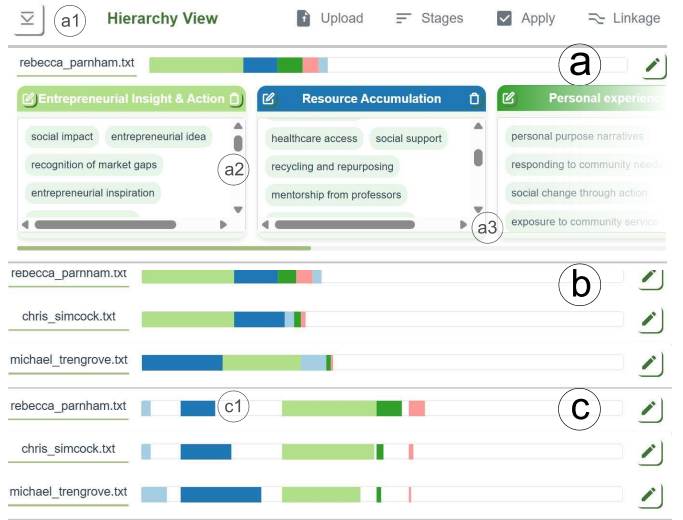


Figure 5: Categorization at the corpus level of the Hierarchy View. (a) Hierarchical coding results of a single article. Colored bars represent second-order codes. (b) After hiding detailed cards, the overall coding results are presented as a stacked bar chart. (c) This mode aligns the same themes vertically, making it easy to compare single second-order codes across all articles.

► **Design Consideration.** We designed the visual representation iteratively. To begin with, we adopted a node-link diagram, as shown in Figure 6(a), where nodes represented first-order codes, with lines at both ends indicating the two codes co-occur sequentially. However, it fails to indicate the hierarchical coding structure. To refine the design, we used convex hulls to enclose first-order codes that belong to the same second-order code, and label each shape with the corresponding second-order code to clearly indicate the hierarchical coding structure, as shown in Figure 6(b). For readability, we performed edge bundling on connecting lines between first-order codes, bundling links belonging to the same pair of second-order codes together to reduce visual clutter. Though presenting the hierarchical coding structure intuitively, experts expressed a need to identify core themes and patterns for theory building. Hence, we further adopted the Sankey diagram to show the prominence of second-order codes, as shown in Figure 6(c). Each rectangle represents a second-order code, and links indicate connections between second-order codes. When a user clicks on a link, the system expands it to show the underlying first-order code relationships, allowing a user to explore the details on demand. Compared to Figure 6(b), this design shows the hierarchical coding structure with an emphasis on second-order codes. We keep both designs (Figure 6 (b) and (c)) in the system, allowing a user to switch their focus between first-order and second-order codes.

◀ The Linkage View introduces two operations to support the provenance of the relationships. To examine the linkage between second-order codes, a user can click to expand a link, revealing the prominent links among the underlying first-order codes (Figure 6(c)). Moreover, through integration with ProvAgent, a user can analyze a certain link in detail. Specifically, a user can right-click a link in the Linkage View to add it to ProvAgent (Figure 1(c1)). They can then analyze the underlying raw text to confirm the validity of this relationship (see Section 4.5).

4.5 ProvAgent: Agentic Provenance Workflow

◀ This view originates from the need for referring to memos during the backward process, which is the analyst’s write-up of emergent ideas about codes and their relationships as they occur during the research process [17]. Now we extend this idea to a more comprehensive agentic workflow, ProvAgent, to support provenance and refinement at scale (T4, Figure 1 (d)). ProvAgent supports four types of memos, i.e., green memo for first-order code (Figure 7(b4)), blue for second-order code (Figure 7(b3)), yellow for theoretical code (Figure 1(d2)), and red for

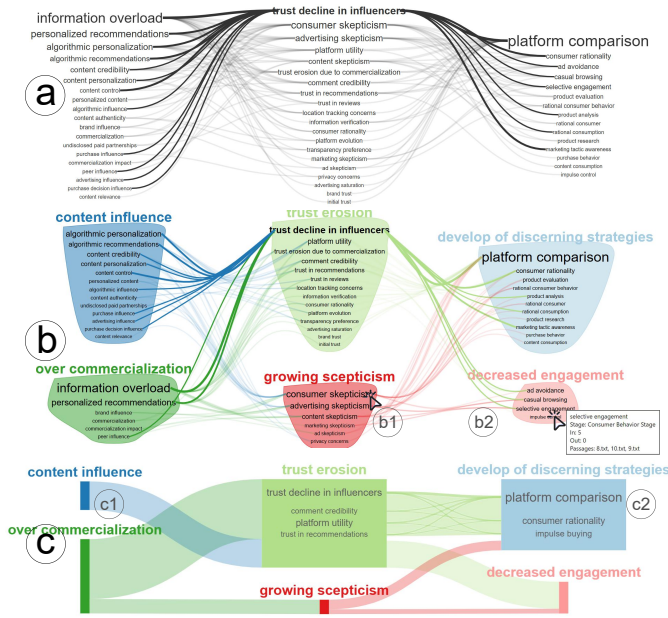


Figure 6: The design alternatives of the Linkage View. (a) The first version of the Linkage View. First-order codes are depicted as nodes, and the link width indicates the frequency of the pattern. (b) The Linkage View emphasizing first-order codes. It employs convex hulls to enclose first-order codes within the same second-order code and applies edge bundling to links between hierarchical codes. (c) The Linkage View with a focus on second-order codes. After a user clicks on a band between second-order codes, the underlying first-order codes and their relationships are displayed. Text color and size both encode the frequency of codes in the corpus.

analytical memos (Figure 1(d3)). By adding a memo, a user can record their overarching thoughts on a specific code or relationship. Also, a user can trigger the agentic workflow to analyze the provenance.

◀ Inspired by “vibe coding” [49], ProVAgent incorporates tool-based planning, reasoning, acting, and observing. The tools follow the function-calling protocol [51], which is applicable to various mainstream LLMs, including DeepSeek-Chat in our implementation. We carefully designed a collection of functions adapted to the codebook. Function details are described in the supplementary material (S3). **Planning & Reasoning:** Upon session initialization, the system loads all available tools into the LLM. When a user inputs a query, the LLM performs task planning and invokes the *READ* tool. The system then receives the invocation, runs the corresponding function, and appends the results to the dialogue, enabling the LLM to reason based on new evidence. **Acting & Observing:** To take actions while ensuring reliability (T5), ProVAgent utilizes a set of pre-defined *WRITE* tools, including merging (combines two second-order codes), splitting (divides a second-order code and redistributes its first-order codes), creating (generates a new code or a memo), deleting (removes a code or a memo), and modifying (updates code names, explanations, user-created memos, conceptual stages, etc), with carefully designed parameters. Before a *WRITE* tool is invoked, the system presents it in the interface (Figure 7(c)) and only updates the codebook upon user confirmation. After executing one operation, the LLM utilizes *READ* tools again to acquire new context, observes the system’s feedback and state changes, and returns to the planning phase to dynamically adjust the next steps. This iterative loop continues until the task is finished or interrupted by a user, as shown in Figure 3.

◀ ProVAgent is deeply coordinated with other views. After a user searches for a first-order code in its memo, the system will extract and list all matching files and source sentences in a unified view (Figure 7b). When a user clicks on a file name, the Content View scrolls to the code block with rich edit capabilities (Figure 1(a)). Similarly, after a second-order code is searched, our system extracts all its associated first-order codes. ProVAgent displays the search results using two

representations: a list and a UMAP [48]. The UMAP displays the semantic distribution of first-order codes. Codes that are semantically similar are positioned close to each other. The size of each point encodes its frequency in the corpus. Color encodes the second-order code to which the first-order code belongs. When a user clicks on a code, either in the list (Figure 7(b3)) or on the UMAP (Figure 7(a3)), the code is displayed in the Content View for further review and edit. Finally, when a user searches for a relationship in ProVAgent, the system reads the codes involved in that relationship, e.g., “volunteer activities with family — peer collaborations,” and conducts a precise global search for documents containing both codes simultaneously (Figure 1(d2)), referring to the Content View for provenance and refinement.

5 CASE STUDY

To evaluate the effectiveness and usability of PROVA-GT in analyzing real-world data, we conducted two case studies in different problem domains, i.e., entrepreneurial management and marketing. Our experiments were approved by our school’s IRB, with the protocol number of ZJUSOM20251224. All experts provided filled in and signed a consent form (supplementary material (S8)) prior to participation. Each interview lasted for about one hour. We demonstrated the system for 15 minutes, then experts freely explored it for 40 minutes while thinking aloud. Finally, a semi-structured interview discussion was conducted to collect the strengths and weaknesses of PROVA-GT. We took notes about their feedback at the same time.

5.1 Analyzing Entrepreneurial Triggers

5.1.1 Case Background

The first case study served as a theoretical exploration to identify new codes and relationships to deepen insights. The data sample consisted of 20 interview transcripts from New Zealand-based social entrepreneurs. Each transcript had an average of 10,000 words. The case study involved three experts, E1-E3, who collaborated with us from the beginning of this research. In this study, E1 acted as the lead domain expert who guided the analysis, while E2 and E3 were the primary operators of the system. They aimed to identify the antecedents, processes, and mechanisms driving entrepreneurial inspiration.

5.1.2 Exploring Entrepreneurs’ Life Events

To leverage LLM for open coding, E2 crafted a prompt in the dialogue panel of the Content View (as shown in the supplementary material (S1, S4)). Then the LLM output a quantity of first-order codes (T1) in the Content View (Figure 1(a)) and the Hierarchy View (Figure 1(b)). E1 said, “In large-scale collaborative coding involving multiple people, maintaining uniform standards becomes difficult. Now (with the help of the LLM), the system is really doing an awesome job in open coding.” However, E1 observed that some codes captured vague emotions rather than specific events, such as “nervousness in front of industry leaders” extracted from “I was nervous when meeting the leader of that organization.” “This code is too vague. Maybe we should revise the coding standards,” E1 noted. E2 therefore refined the prompt to emphasize concrete triggers. E1 further explained, “With (the help of) this system, we can quickly see the effects of different coding standards and iterate.” Finally, PROVA-GT generated 450 first-order codes.

Next, experts sought to cluster first-order codes into second-order codes. E2 clicked the edit button of the first file and moved to the file level of the Hierarchy View. She noticed that terms such as “learning from failure” stood out from the word cloud (Figure 4(a)). E2 then created a card named “personal experiences” (Figure 4(b)) and clicked the “absorb” button on the card. The system then automatically moved relevant codes into that card (Figure 4(d)). “The results look nice. However, some (first-order) codes are not relevant (to personal experiences)” E1 said. Subsequently, E2 moved some codes out of the card, such as “adaptability.” After that, E2 created a set of cards across multiple transcripts, including “resource accumulation”, “entrepreneurial insights and actions”, etc. Next, E2 turned to the corpus level of the Hierarchy View (Figure 5(a)) and applied the coding scheme to the corpus (as shown in the supplementary material (S2)). To have an overview of the coding results across files, E2 clicked the

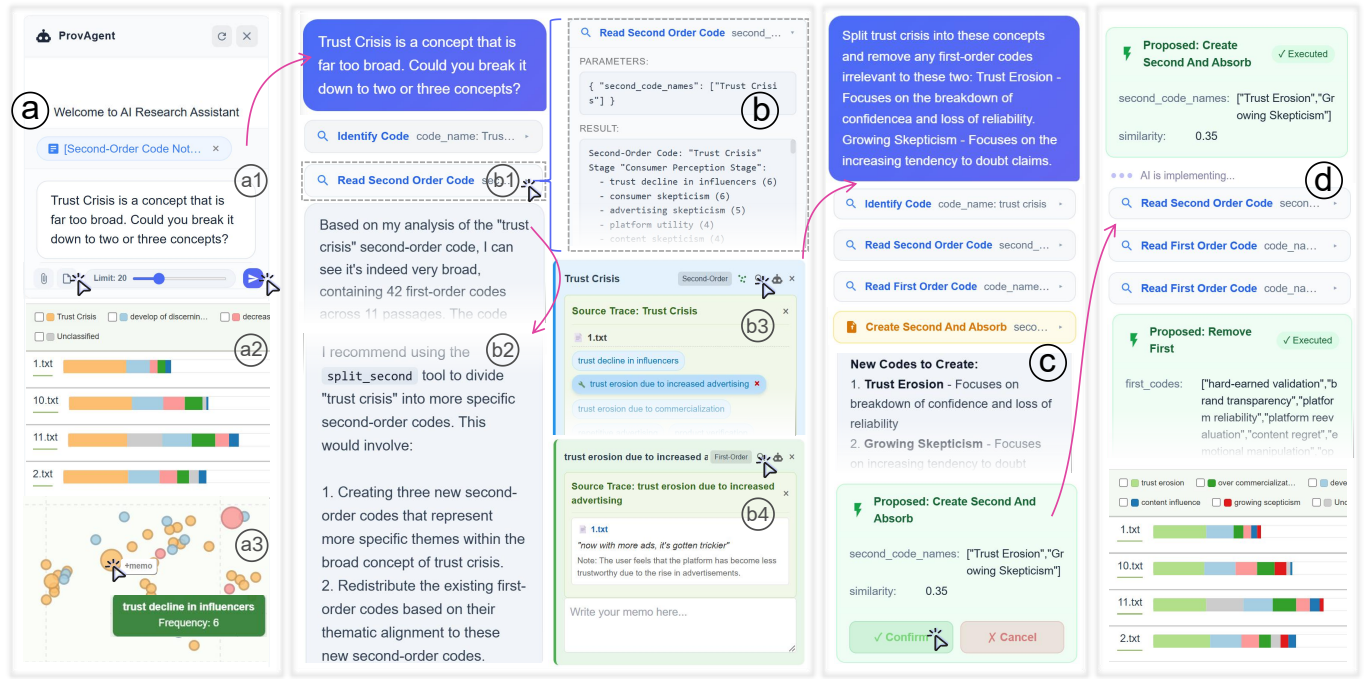


Figure 7: The iterative workflow of ProvAgent for backward analysis. (a) A user seeks to break a second-order code into two by starting a query in natural language. (b) ProvAgent enters the planning phase, invokes *READ* tools, and reasons over the retrieved data to generate action proposals. (c) Subject to user confirmation, ProvAgent executes *WRITE* tools to apply changes to the codebook. (d) ProvAgent observes the state changes and returns to the planning phase, continuing the iterative refinement.

“stack” button in the Hierarchy View (Figure 5(a1)). Then all cards were hidden, leaving a stacked bar chart displayed (Figure 5(b)). Experts observed the distribution of codes in each transcript and compared the proportions of themes among different transcripts (Figure 5(c)). E2 commented, “The workflow is clear, and I can maintain control over the entire coding process.” (T2)

5.1.3 Constructing Entrepreneurial Motivation Theory

After establishing the hierarchical codes, E3 took control to directly explore how concepts transitioned between codes (T3), so she turned to the Linkage View (Figure 1(c)). E3 first saw many first-order codes enclosed by convex hulls and distributed across various stages. “The information explodes.” E1 exclaimed. To distill core relationships, E3 raised the path threshold to five, filtering the most frequent relationships. For example, the code “early leadership experiences” under “early contextual triggers” during the childhood stage were linked to “peer collaborations” (Figure 1(c1)). By hovering over the link, she discovered that this relationship appeared in seven articles and viewed the specific article titles. “This is a significant pattern,” E1 noted, “likely because, as Bourdieu’s theory suggests, our earliest social experiences lay the foundation for the habitus that shapes our lifelong behaviors.”

Next, E3 sought to further investigate the link between “volunteer activities with family” and “peer collaborations”. Recalling the literature *Social Entrepreneurship: How Intentions to Create a Social Venture are Formed* [46], which states that social entrepreneurship involves the “innovative use of resource combinations to pursue opportunities”, she manually verified the source trace via a theoretical memo (Figure 1(d2)) (T4). She queried ProvAgent by attaching the literature and relevant code notes (Figure 1(d1)), asking how their interplay shapes social entrepreneurial identity. ProvAgent synthesized the findings into an Analytical Memo. E3 highly praised this design, “I can coherently verify relationships without tedious manual comparisons back in the code.” She further added, “The agent’s transparent use of tools keeps me in control and deepens my trust in the AI’s outputs.” (T5) Finally, the experts proposed a theoretical framework that summarized the relationships (Figure 1(c)). The framework revealed that early triggers and entrepreneurial action are separated by a time lag, mediated

by cognitive processes. Detailed descriptions of the framework are presented in Supplementary Material (S5).

5.2 Examining Trust Erosion in Commercialization

5.2.1 Case Background

The second case study aimed to examine whether the results identified from PROVA-GT were consistent with the preliminary findings in manual coding. The dataset comprised 11 interview transcripts from Rednote users, collected in October 2024, with an average of 8,000 words each. These narratives detailed user experiences with influencer content amid the growing commercialization of Rednote. A professor in marketing (E4) and her postgraduate student (E5) participated in the evaluation phase. They sought to elucidate the antecedents, processes, and mechanisms underlying trust erosion driven by influencer behavior.

5.2.2 Constructing Trust Erosion Theory

Similar to the operations in the first case study, E5 finished open coding and categorization (T1). Before exploring relationships, E5 noticed from the stacked bar chart and UMAP (Figure 7(a2), (a3)) that the second-order code “Trust Crisis” was overly broad. Initiating a dialogue with ProvAgent (Figure 7(a1)), she asked: “Trust Crisis is a concept that is far too broad. Could you break it down to two or three concepts?” ProvAgent invoked *READ* tools (Figure 7(b1)) and recommended the “split second” tool (Figure 7(b2)). While reviewing the “Trust Crisis” source trace board and underlying first-order codes like “trust erosion due to increased advertising,” E5 commanded: “Split trust crisis into Trust Erosion and Growing Scepticism, and remove any first-order codes irrelevant to these two.” (T2) ProvAgent then invoked *READ* tools again, proposed a Create Second And Absorb action for the new codes, and executed it (Figure 7(c)). Observing the task was incomplete, the agent subsequently proposed and executed a Remove First action to clean up the irrelevant codes (Figure 7(d)). E5, excited about ProvAgent, said, “It is too efficient (to believe).” (T5)

To explore relationships within the trust erosion process, E5 switched to the Linkage View (Figure 6(b), T3). She aimed to investigate a hypothesis: whether growing scepticism leads to active changes in user behavior. To test this, she clicked on the “consumer scepticism” node (Figure 6(b1)). The view highlighted its connections, revealing

a particularly strong link to “platform comparison.” While exploring the flows, E5 noticed some nodes were misplaced, such as “selective engagement.” She right-clicked the node and reassigned it to the “decreased engagement” category (Figure 6(b2)). E5 commented, “Now I need to focus on the most significant patterns.” She then increased the threshold to four to filter out less common patterns (as shown in the supplementary material (S6)). Further, she also used the “combine” button to aggregate related nodes and links (Figure 6(c)). She clicked the band connecting “trust erosion” and “develop of discerning strategies,” observing a thick, significant connection directly between “platform comparison” and “trust decline in influencers” (Figure 6(c2)). “This is exactly the connection we theorized,” she said (T4). Ultimately, experts used the Linkage View to validate her findings on trust erosion, presenting the resulting model (Figure 6(b)(c)). “Trust erosion begins with influencer behavior (Figure 6(c1)), leading to increasing consumer scepticism (Figure 6(b1)). This scepticism fosters discerning strategies and decreased engagement.” (Figure 6(c2)) E4 stated.

5.3 Expert Feedback and Insights

Experts gave positive feedback on the effectiveness and efficiency of PROVA-GT. Also, the experts provided feedback on the usability issues and offered suggestions for future enhancements. For instance, after completing her coding, E1 hoped to extend the coding scheme to a new, similar corpus with a single click. She commented, “I hope the system can learn from my coding behavior and automatically extend the scheme when new interview transcripts are introduced.” E1 also mentioned that although PROVA-GT supports importing and exporting codebooks, it currently lacked version management. As E1 noted, “Sometimes I just want to do some exploration, so I need to go back to the previous state of the codebook.” Furthermore, E2 complained, “when applying the coding scheme to all the articles, it took about three minutes.” She added, “given 20 transcripts, I feel that the time is too long.” In addition, E4 suggested, “It would be awesome if the system introduced computational methods to help us discover correlations or causality patterns.” Moreover, she brainstormed a new direction of our system, “The system could proactively detect when I’m stuck and offer data-driven insights.” This kind of proactive help would make the whole research process way more interesting and probably lead to some new ideas or meaningful discoveries I wouldn’t have found on my own.” We plan to address these points in future improvements.

6 USER STUDY

To quantify the effectiveness of ProvAgent in supporting provenance, we conducted a controlled lab study comparing our system to NVivo [23], a well-known commercial candidate widely used for qualitative coding. We chose NVivo because it is one of the established software programs that supports the entire GT workflow. We recruited 16 participants (8 males and 8 females, aged 21–23). All were undergraduate or graduate students in a business school, majoring in business administration, information management, and management science. All of them had basic background knowledge in qualitative coding methods, such as thematic analysis or grounded theory, through relevant coursework or previous research experience. The study followed a between-subjects design. Participants were randomly assigned to one of the two groups: the **PROVA-GT group** (P1–P8) and the **NVivo group** (P9–P16), balanced by education level and prior coding experience. Participants received \$15 as compensation.

6.1 Tasks

The task was to analyze four interview transcripts regarding social entrepreneurs (each approximately 1,000 words) to understand their entrepreneurial inspirations and motivations. The transcripts were provided by E1, who had previously coded them manually. The tasks were designed according to two criteria. First, they should originate from real-world coding scenarios, ensuring they were representative of the GT workflow. Second, they should cover a wide range of key steps of the workflow, including open coding, categorization, and theorization. Finally, we derived five tasks from the discussion with E1. That is, evaluate the validity of 1) first-order codes, 2) second-order codes,

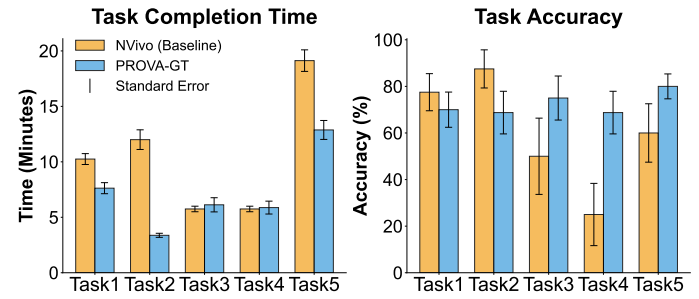


Figure 8: Task performance results in terms of completion time and accuracy across different tasks.

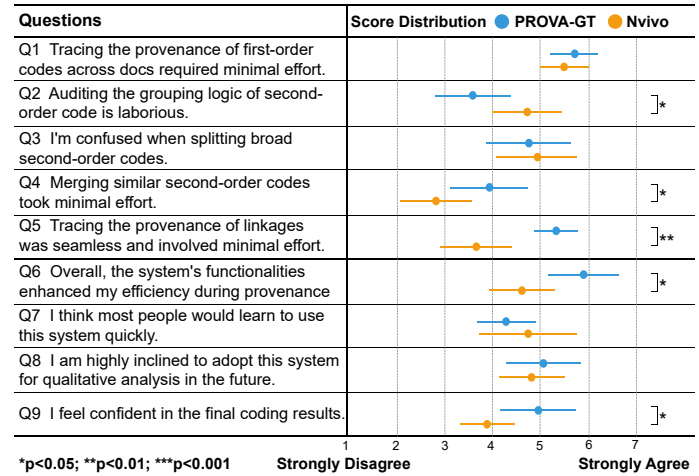


Figure 9: The questionnaire for the user study, including questions on task effort (Q1-Q5) and system utility and usability (Q6-Q9).

3) separation of second-order codes, 4) combination of second-order codes, and 5) linkages between codes. All tasks have ground truth answers based on E1’s manual coding results. Task details are shown in the supplementary material (S7).

6.2 Procedure

Our experiments were approved by our school’s IRB, with the protocol number of ZJUSOM20251224. We obtained informed consent from the participant, who filled in and signed a consent form (supplementary material (S8)). Each experiment began with a 5-minute introduction followed by 15 minutes of standardized training on the assigned tool (NVivo or PROVA-GT). Participants then performed the five coding tasks. Subsequently, they completed a 7-point Likert scale questionnaire guided by the System Usability Scale [12] to assess the utility and usability of the system (Figure 9). Finally, we conducted a 15-minute semi-structured interview to collect qualitative feedback. The entire procedure lasted approximately 80 minutes for each participant. We recorded the screen and audio for later analysis.

6.3 Results — Task Performance

For each task, we analyzed the completion time and accuracy jointly (Figure 8) to assess the performance of each system. In **Task 1**, the PROVA-GT group showed a shorter completion time than the NVivo group. This was because ProvAgent coordinated closely with the Content View, providing a unified interface for the provenance process. On the contrary, NVivo required a user to manually switch between tabs to access different transcripts and the codebook, which was time-consuming. In terms of accuracy, the PROVA-GT group was comparable to the NVivo group. In **Task 2**, the completion time of the PROVA-GT group was significantly shorter than that of the NVivo group. This was because the PROVA-GT group utilized the agentic workflow to collect all first-order codes associated with a second-order code. However, the NVivo group had to manually check the codebook

and trace back to the original text for each first-order code. Interestingly, the accuracy of the PROVA-GT group was lower than that of the NVivo group. After analyzing the screen recording, we noticed that ProvAgent sometimes read the first-order codes without their explanations and text fragments, presenting insufficient context to participants. This was because task planning might not work accurately in some cases, which was a limitation of the prototype. The results of **Tasks 3 and 4** presented a different picture. Both groups took comparable time to complete the tasks, but the PROVA-GT group achieved significantly higher accuracy than the NVivo group. The two tasks required a deep understanding of both the semantic discrepancy between the first and second-order codes, and the raw text linking to the first-order codes. The PROVA-GT group utilized ProvAgent to gather all the necessary information. The NVivo group, however, made the decision primarily based on the codebook, without verifying the source text for each first-order code. **Task 5** was the most challenging, requiring understanding of code pairs and their context. The NVivo group manually scrolled through transcripts to locate relevant text, while the PROVA-GT group leveraged the agentic workflow to collect transcripts and provide reliable suggestions. Hence, the PROVA-GT group excelled in both speed and accuracy in this task.

6.4 Results — Questionnaire and Qualitative Feedback

As shown in Figure 9, we evaluated the mental demand for each task (Q1–Q5). Compared to NVivo, participants using PROVA-GT reported significantly less mental effort when discerning discrepancies for categorization (Q2, $p < 0.05$), auditing the code hierarchy for structural errors (Q4, $p < 0.05$), and tracing the provenance of theoretical links (Q5, $p < 0.01$). In terms of Q1 and Q3, participants using PROVA-GT reported less confusion on average, though these differences did not reach statistical significance. Participants felt that PROVA-GT significantly enhanced their overall efficiency during the provenance and refinement process compared to NVivo (Q6, $p < 0.05$). Furthermore, participants reported significantly higher confidence in their final analytical results when using PROVA-GT (Q9, $p < 0.05$). In terms of accessibility (Q7), NVivo scored slightly higher than PROVA-GT, though the difference was not significant. This was expected, because the agentic workflow was new to a user, while NVivo’s interface was more familiar to those with prior coding experience. Despite the learning curve, participants expressed a stronger inclination to adopt PROVA-GT for future qualitative analysis (Q8).

During the semi-structured interviews, PROVA-GT received enthusiastic responses from participants, especially regarding its AI-assisted workflow. They also provided constructive feedback for future improvements. First, participants suggested that ProvAgent provide reference links in the responses. As P3 noted, “When evaluating the AI’s response, I would like to know the information it relies on.” “Like the Deep Research Mode in Gemini,” P3 added. Second, P5 suggested that the system provide session management features for ProvAgent, “After doing a lot of tasks, the chat dialog becomes too long.” Third, some participants, including P1, P9, and P12, raised several usability issues. One common issue was the latency when calling the LLM. P12 said, “Sometimes I have to wait for a long time after a query.”

7 DISCUSSION

7.1 The Use of LLM in the GT Workflow

Employing the LLM in a GT workflow requires caution. Since experts have to verify the entire codebook — whether generated by the LLM or manually — to ensure the validity of the theorization, automatically generating massive codes may place a significant burden on experts during verification. As a result, this paper focuses on provenance and refinement rather than code generation. The user study revealed that the key to the success of ProvAgent was to provide sufficient context for a user to validate the coding results with minimal effort.

On the other hand, the user study (Task 2) also revealed that the agentic workflow could miss important context due to inaccurate task planning. To be specific, before fetching relevant context, the LLM first proposes a sequence of actions, where the first action, i.e., call READ tools, determines which information to fetch. Since the LLM output is random and suffers from hallucination to some extent, it

may occasionally fail to retrieve all relevant information in some cases. When a user relies heavily on an LLM for information fetching without sufficient scrutiny, the information presented in ProvAgent becomes misleading, thereby leading to judgment errors. We plan to mitigate this risk in terms of three aspects. First, we will explore retrieval-augmented generation techniques to query context by semantics, presenting a broader range of relevant information. Second, since a single LLM may make mistakes, we will explore the use of multiple LLMs to cross-validate the proposed actions as well as the retrieved context. Third, user confirmation serves as the last line of defense. We plan to present some visual-aids, such as UMAP, to help a user better understand the semantic distribution of a given code and its context. By doing so, a user can critically review and reflect on the context.

7.2 Limitations and Future Work

We acknowledge three limitations in system design and implementation, and outline potential directions for future research. First, the current Linkage View reveals frequent patterns between concepts, leaving correlation and causal relationships under-exploited. Besides integrating computational methods to uncover various relationships, we plan to investigate visualization techniques to facilitate a user in identifying and auditing these relationships. Second, the Conceptual Stage mechanism was proposed by our domain experts and assumes a temporal ordering of stages. It may not be applicable to situations where phases are non-temporal or concepts are not organized by stages. Future work could adopt a graph model, allowing users to flexibly define the semantics of edges between codes. We plan to transform the current Sankey-based layout into a node-link representation suitable for diverse GT tasks. Third, the integration of LLM has introduced computational overhead, resulting in delayed feedback to the end user. We plan to stream partial results as the LLM processes.

The study design can be improved in terms of three aspects. First, the participants involved in the user study do not align with the target users of PROVA-GT, who are social science researchers. In addition, the study used four short interview transcripts with simplified coding tasks, which are different from real coding scenario involving lengthy transcripts and iterative coding process. This limits the generalizability of the quantitative results to expert-level practice. Second, NVivo does not employ LLMs in validation, as Table 1 shows in the “LLM-based reasoning” column. Hence, the study reveals the effectiveness of PROVA-GT in comparison to a baseline without LLM support. We plan to compare PROVA-GT with systems facilitated by LLM-based validation, such as ATLAS.ti and MAXQDA, in follow-up studies. Third, the case study involves experts in two domains, i.e., entrepreneurial management and marketing. To validate the broader applicability of PROVA-GT, future research should extend the application to other domains, such as healthcare and education.

7.3 Generalizability

Though PROVA-GT is guided by grounded theory and design tasks derived from experts in marketing and entrepreneurial management, its core design is broadly generalizable to diverse applications. For example, the functionalities offered by the Content View and Hierarchy View can support Content Analysis [8], which involves coding and quantitatively analyzing text data to identify keywords and trends. Another example is that ProvAgent, the agentic workflow incorporating the READ and WRITE tools, along with the iterative planning, reasoning, acting, and observing processes, can be applied to systems that require provenance tracing. By linking with other views, ProvAgent provides a unified interface to present evidence for decision-making.

8 CONCLUSION

In this paper, we present a two-year design study with experts in entrepreneurial management, and develop a visual analytics system, PROVA-GT, that supports grounded theory. By incorporating an agentic workflow, PROVA-GT is superior in facilitating the backward process of verifying and refining coding results. Results from two case studies and a lab study demonstrated the effectiveness and usefulness of PROVA-GT in supporting theory building and code verification.

ACKNOWLEDGMENTS

The authors wish to thank anonymous reviewers. This work is supported by the National Natural Science Foundation of China (Grant No.72192823) and the Zhejiang Key Laboratory of Decision Intelligence (Grant No. 2025E10006).

REFERENCES

- [1] R. Agrawal and R. Srikant. Mining sequential patterns. In *Proceedings of the 11th International Conference on Data Engineering*, pp. 3–14, 1995. doi: 10.1109/ICDE.1995.380415 5
- [2] E. Alexander, J. Kohlmann, R. Valenza, M. Witmore, and M. Gleicher. Serendip: Topic model-driven visual exploration of text corpora. In *Proceedings of the IEEE Conference on Visual Analytics Science and Technology*, pp. 173–182, 2014. doi: 10.1109/VAST.2014.7042493 2, 3
- [3] L. Alqazlan, Z. Fang, M. Castelle, and R. Procter. A novel, human-in-the-loop computational grounded theory framework for big social data. *Big Data & Society*, 12(2), 2025. doi: 10.1177/20539517251347598 1
- [4] P. Alvaro, J. Rosen, and J. M. Hellerstein. Lineage-driven fault injection. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 331–346, 2015. doi: 10.1145/2723372.2723711 2
- [5] ATLAS.ti Scientific Software Development GmbH. ATLAS.ti: Qualitative data analysis platform. <https://atlasti.com/>, 2025. Accessed: Jun 10, 2026. 2, 3
- [6] C. F. Barros, B. B. Azevedo, V. V. Graciano Neto, M. Kassab, M. Kalinowski, H. A. D. Do Nascimento et al. Large language model for qualitative research: A systematic mapping study. In *Proceedings of the IEEE/ACM International Workshop on Software Engineering for Science and Engineering*, pp. 48–55, 2025. doi: 10.1109/WSESE66602.2025.00015 1
- [7] E. P. S. Baumer, D. Siedel, L. McDonnell, J. Zhong, P. Sittikul, and M. McGee. Topicalizer: Reframing core concepts in machine learning visualization by co-designing for interpretivist scholarship. *Human-Computer Interaction*, 35(5-6):452–480, 2020. doi: 10.1080/07370024.2020.1734460 2
- [8] B. Berelson. *Content Analysis in Communication Research*. Free Press, 1952. 9
- [9] A. Biswas, T. L. Turton, N. R. Ranasinghe, S. Jones, B. Love, W. Jones et al. VizGenie: Toward self-refining, domain-aware workflows for next-generation scientific visualization. *IEEE Transactions on Visualization and Computer Graphics*, 32(1):1021–1031, 2026. doi: 10.1109/TVCG.2025.3634655 2
- [10] D. M. Blei and J. D. Lafferty. Topic models. In *Text Mining: Classification, Clustering, and Applications*, pp. 71–94. Chapman and Hall/CRC, 2009. doi: 10.1201/9781420059458.ch4 2
- [11] M. A. Borkin, C. S. Yeh, M. Boyd, P. Macko, K. Z. Gajos, M. Seltzer et al. Evaluation of filesystem provenance visualization tools. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2476–2485, 2013. doi: 10.1109/TVCG.2013.155 2
- [12] J. Brooke. SUS: A retrospective. *Journal of Usability Studies*, 8(2):29–40, 2013. doi: 10.5555/2817912.2817913 8
- [13] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, pp. 1877–1901, 2020. doi: 10.48550/arXiv.2005.14165 2
- [14] X. Cai, X. Ge, K. Xiong, S. Ye, D. Weng, K. Xu et al. HYPNOS: Interactive data lineage tracing for data transformation scripts. *IEEE Transactions on Visualization and Computer Graphics*, 31(10):7632–7646, 2025. doi: 10.1109/TVCG.2025.3552091 2
- [15] S. P. Callahan, J. Freire, E. Santos, C. E. Scheidegger, C. T. Silva, and H. T. Vo. VisTrails: Visualization meets data management. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 745–747, 2006. doi: 10.1145/1142473.1142574 2
- [16] T.-C. Chang, A. R. P. Li, C.-Y. Wang, and J. J. H. Lin. From automation to thinking: The role of AGI in discourse analysis of computer-supported collaborative learning based on computational grounded theory. *Computers & Education*, 247:105579, 2026. doi: 10.1016/j.compedu.2026.105579 1, 2
- [17] K. Charmaz and L. L. Belgrave. Grounded theory. In *The Blackwell Encyclopedia of Sociology*, pp. 1–6. Wiley Online Library, 2007. doi: 10.1002/9781405165518.wbeosg070.pub2 1, 5
- [18] R. Chew, J. Bollenbacher, M. Wenger, J. Speer, and A. Kim. LLM-assisted content analysis: Using large language models to support deductive coding. *arXiv preprint arXiv:2306.14924*, 2023. doi: 10.48550/arXiv.2306.14924 1
- [19] J. Choo, C. Lee, C. K. Reddy, and H. Park. UTOPIAN: User-driven topic modeling based on interactive nonnegative matrix factorization. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):1992–2001, 2013. doi: 10.1109/TVCG.2013.212 2
- [20] R. Churchill and L. Singh. The evolution of topic modeling. *ACM Computing Surveys*, 54(10s), art. no. 215, 35 pages, 2022. doi: 10.1145/3507900 2
- [21] S.-C. Dai, A. Xiong, and L.-W. Ku. LLM-in-the-loop: Leveraging large language model for thematic analysis. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 9993–10001, 2023. doi: 10.18653/v1/2023.findings-emnlp.669 2
- [22] S. De Paoli and W. S. Mathis. Reflections on inductive thematic saturation as a potential metric for measuring the validity of an inductive thematic analysis with LLMs. *Quality & Quantity*, 59(1):683–709, 2024. doi: 10.1007/s11135-024-01950-6 1
- [23] K. Dhakal. NVivo. *Journal of the Medical Library Association*, 110(2):270–272, 2022. doi: 10.5195/jmla.2022.1271 1, 2, 3, 8
- [24] V. Dibia. LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 113–126, 2023. doi: 10.18653/v1/2023.acl-demo.11 2
- [25] A. Diehl, A. Abdul-Rahman, B. Bach, M. El-Assady, M. Kraus, R. S. Laramée et al. An analysis of the interplay and mutual benefits of grounded theory and visualization. *IEEE Transactions on Visualization and Computer Graphics*, 31(9):5462–5479, 2025. doi: 10.1109/TVCG.2024.3452985 3
- [26] M. El-Assady, R. Kehlbeck, C. Collins, D. A. Keim, and O. Deussen. Semantic Concept Spaces: Guided topic model refinement using word-embedding projections. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1001–1011, 2020. doi: 10.1109/TVCG.2019.2934654 2, 3
- [27] Equafin Corp. Hey Marvin: Qualitative data analysis platform. <https://heymarvin.com/>, 2020. Accessed: Mar 29, 2026. 2, 3
- [28] J. L. Feuston and J. R. Brubaker. Putting tools in their place: The role of time and perspective in human-AI collaboration for qualitative analysis. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW2), art. no. 469, 25 pages, 2021. doi: 10.1145/3479856 2
- [29] J. Gao, K. T. W. Choo, J. Cao, R. K.-W. Lee, and S. Perrault. CoAlcoder: Examining the effectiveness of AI-assisted human-to-human collaboration in qualitative analysis. *ACM Transactions on Computer-Human Interaction*, 31(1), art. no. 6, 38 pages, 2023. doi: 10.1145/3617362 1, 2
- [30] J. Gao, Y. Guo, G. Lim, T. Zhang, Z. Zhang, T. J.-J. Li et al. CollabCoder: A lower-barrier, rigorous workflow for inductive collaborative qualitative analysis with large language models. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, art. no. 11, 29 pages, 2024. doi: 10.1145/3613904.3642002 2, 3
- [31] L. E. George and L. Birla. A study of topic modeling methods. In *Proceedings of the 2nd International Conference on Intelligent Computing and Control Systems*, pp. 109–113, 2018. doi: 10.1109/ICCONS.2018.8663152 2
- [32] D. A. Gioia, K. G. Corley, and A. L. Hamilton. Seeking qualitative rigor in inductive research: Notes on the Gioia methodology. *Organizational Research Methods*, 16(1):15–31, 2013. doi: 10.1177/1094428112452151 3
- [33] B. G. Glaser. *Doing Grounded Theory: Issues and Discussions*. Sociology Press, 1998. doi: 10.4135/9781529716658 1
- [34] B. G. Glaser and A. L. Strauss. *Discovery of Grounded Theory: Strategies for Qualitative Research*. Aldine, 1967. doi: 10.4324/9780203793206 1
- [35] M. Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794*, 2022. doi: 10.48550/arXiv.2203.05794 2
- [36] R. Hoekstra and P. Groth. PROV-O-Viz – understanding the role of activities in provenance. In *Proceedings of the 5th International Workshop on Provenance and Annotation of Data and Processes*, pp. 215–220, 2014. doi: 10.1007/978-3-319-16462-5_18 2
- [37] M.-H. Hong, L. A. Marsh, J. L. Feuston, J. Ruppert, J. R. Brubaker, and D. A. Szafir. Scholastic: Graphical human-AI collaboration for inductive

- and interpretive text analysis. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, art. no. 30, 12 pages, 2022. doi: [10.1145/3526113.3545681](https://doi.org/10.1145/3526113.3545681) 2, 3
- [38] M. S. Jalali and A. Akhavan. Integrating AI language models in qualitative research: Replicating interview data analysis with ChatGPT. *System Dynamics Review*, 40(3):e1772, 2024. doi: [10.1002/sdr.1772](https://doi.org/10.1002/sdr.1772) 1
- [39] J. A. Jiang, K. Wade, C. Fiesler, and J. R. Brubaker. Supporting serendipity: Opportunities and challenges for human-AI collaboration in qualitative analysis. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW1):1–23, 2021. doi: [10.1145/3449168](https://doi.org/10.1145/3449168) 1
- [40] H. Kim, B. Drake, A. Ender, and H. Park. ArchiText: Interactive hierarchical topic modeling. *IEEE Transactions on Visualization and Computer Graphics*, 27(9):3644–3655, 2021. doi: [10.1109/TVCG.2020.2981456](https://doi.org/10.1109/TVCG.2020.2981456) 2
- [41] J. Klaus, M. Blacher, A. Goral, P. Lucas, and J. Giesen. A visual analytics workflow for probabilistic modeling. *Visual Informatics*, 7(2):72–84, 2023. doi: [10.1016/j.visinf.2023.05.001](https://doi.org/10.1016/j.visinf.2023.05.001) 2
- [42] U. Kuckartz and S. Rädiker. *Analyzing Qualitative Data with MAXQDA*. Springer, 2019. doi: [10.1007/978-3-030-15671-8](https://doi.org/10.1007/978-3-030-15671-8) 2, 3
- [43] U. Kuckartz and S. Rädiker. *Focused Analysis of Qualitative Interviews with MAXQDA*. MAXQDA Press, 2020. doi: [10.36192/978-3-948768072](https://doi.org/10.36192/978-3-948768072) 2
- [44] M. S. Lam, J. Teoh, J. A. Landay, J. Heer, and M. S. Bernstein. Concept induction: Analyzing unstructured text with high-level concepts using LLoOM. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, art. no. 766, 28 pages, 2024. doi: [10.1145/3613904.3642830](https://doi.org/10.1145/3613904.3642830) 2, 3
- [45] S. Liu, M. X. Zhou, S. Pan, Y. Song, W. Qian, W. Cai et al. TIARA: Interactive, topic-based visual text summarization and analysis. *ACM Transactions on Intelligent Systems and Technology*, 3(2), art. no. 25, 28 pages, 2012. doi: [10.1145/2089094.2089101](https://doi.org/10.1145/2089094.2089101) 2
- [46] J. Mair and E. Noboa. Social entrepreneurship: How intentions to create a social venture are formed. In *Social Entrepreneurship*, pp. 125–135. Palgrave Macmillan UK, 2006. doi: [10.1057/9780230625655_8](https://doi.org/10.1057/9780230625655_8) 7
- [47] M. Marathe and K. Toyama. Semi-automated coding for qualitative research: A user-centered inquiry and initial prototypes. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1–12, 2018. doi: [10.1145/3173574.3173922](https://doi.org/10.1145/3173574.3173922) 2
- [48] L. McInnes, J. Healy, N. Saul, and L. Großberger. UMAP: Uniform manifold approximation and projection for dimension reduction. *Journal of Open Source Software*, 3(29):861, 2018. doi: [10.21105/joss.00861](https://doi.org/10.21105/joss.00861) 6
- [49] C. Meske, T. Hermanns, E. Von der Weiden, K.-U. Loser, and T. Berger. Vibe coding as a reconfiguration of intent mediation in software development: Definition, implications, and research agenda. *IEEE Access*, 13:213242–213259, 2025. doi: [10.1109/ACCESS.2025.3645466](https://doi.org/10.1109/ACCESS.2025.3645466) 2, 6
- [50] D. L. Morgan. Exploring the use of artificial intelligence for qualitative data analysis: The case of ChatGPT. *International Journal of Qualitative Methods*, 22:16094069231211248, 2023. doi: [10.1177/16094069231211248](https://doi.org/10.1177/16094069231211248) 1
- [51] OpenAI. Function calling. <https://developers.openai.com/api/docs/guides/function-calling>, 2026. Accessed: Jun 20, 2026. 6
- [52] R. Pan, Y. Wang, J. Sun, H. Liu, Y. Zhao, J. Xia et al. Simplifying social networks via triangle-based cohesive subgraphs. *Visual Informatics*, 7(4):84–94, 2023. doi: [10.1016/j.visinf.2023.07.003](https://doi.org/10.1016/j.visinf.2023.07.003) 2
- [53] R. Ramalho, P. Adams, P. Huggard, and K. Hoare. Literature review and constructivist grounded theory methodology. *Forum: Qualitative Social Research*, 16(3):Art. 19, 2015. doi: [10.17169/fqs-16.3.2313](https://doi.org/10.17169/fqs-16.3.2313) 2
- [54] H. Schroeder, M. Aubin Le Quééré, C. Randazzo, D. Mimno, and S. Schoenebeck. Large language models in qualitative research: Uses, tensions, and intentions. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, art. no. 481, 17 pages, 2025. doi: [10.1145/3706598.3713120](https://doi.org/10.1145/3706598.3713120) 1
- [55] M. Sedlmair, M. Meyer, and T. Munzner. Design study methodology: Reflections from the trenches and the stacks. *IEEE Transactions on Visualization and Computer Graphics*, 18(12):2431–2440, 2012. doi: [10.1109/TVCG.2012.213](https://doi.org/10.1109/TVCG.2012.213) 3
- [56] H. Stitz, S. Luger, M. Streit, and N. Gehlenborg. AVOCADO: Visualization of workflow-derived data provenance for reproducible biomedical research. *Computer Graphics Forum*, 35(3):481–490, 2016. doi: [10.1111/cgf.12924](https://doi.org/10.1111/cgf.12924) 2
- [57] H. Theelen, J. Vreuls, and J. Rutten. Doing research with help from ChatGPT: Promising examples for coding and inter-rater reliability. *International Journal of Technology in Education*, 7(1):1–18, 2024. doi: [10.46328/ijte.537](https://doi.org/10.46328/ijte.537) 1
- [58] M. G. Torii, T. Murakami, and Y. Ochiai. Expanding horizons in HCI research through LLM-driven qualitative analysis. *arXiv preprint arXiv:2401.04138*, 2024. doi: [10.48550/arXiv.2401.04138](https://doi.org/10.48550/arXiv.2401.04138) 2
- [59] T. Übellacker. AcademiaOS: Automating grounded theory development in qualitative research with large language models. *arXiv preprint arXiv:2403.08844*, 2024. doi: [10.48550/arXiv.2403.08844](https://doi.org/10.48550/arXiv.2403.08844) 2, 3
- [60] J. A. Wasserman, J. M. Clair, and K. L. Wilson. Problematics of grounded theory: Innovations for developing an increasingly rigorous qualitative method. *Qualitative Research*, 9(3):355–381, 2009. doi: [10.1177/1468794109106605](https://doi.org/10.1177/1468794109106605) 1
- [61] L. Weng, X. Wang, J. Lu, Y. Feng, Y. Liu, H. Feng et al. InsightLens: Augmenting LLM-powered data analysis with interactive insight management and navigation. *IEEE Transactions on Visualization and Computer Graphics*, 31(06):3719–3732, 2025. doi: [10.1109/TVCG.2025.3567131](https://doi.org/10.1109/TVCG.2025.3567131) 2
- [62] C. Weston, T. Gandell, J. Beauchamp, L. McAlpine, C. Wiseman, and C. Beauchamp. Analyzing interview data: The development and evolution of a coding system. *Qualitative Sociology*, 24(3):381–400, 2001. doi: [10.1023/A:1010690908200](https://doi.org/10.1023/A:1010690908200) 3
- [63] Z. Xiao, X. Yuan, Q. V. Liao, R. Abdelghani, and P.-Y. Oudeyer. Supporting qualitative analysis with large language models: Combining codebook with GPT-3 for deductive coding. In *Companion Proceedings of the 28th International Conference on Intelligent User Interfaces*, pp. 75–78, 2023. doi: [10.1145/3581754.3584136](https://doi.org/10.1145/3581754.3584136) 1
- [64] K. Xiong, S. Fu, G. Ding, Z. Luo, R. Yu, W. Chen et al. Visualizing the scripts of data wrangling with SOMNUS. *IEEE Transactions on Visualization and Computer Graphics*, 29(6):2950–2964, 2023. doi: [10.1109/TVCG.2022.3144975](https://doi.org/10.1109/TVCG.2022.3144975) 2
- [65] M. Yu, Y. Wang, X. Yu, G. Shan, and Z. Jin. PubExplorer: An interactive analytical system for visualizing publication data. *Visual Informatics*, 7(3):65–74, 2023. doi: [10.1016/j.visinf.2023.07.001](https://doi.org/10.1016/j.visinf.2023.07.001) 2
- [66] Y. Yue, D. Liu, Y. Lv, J. Hao, and P. Cui. A practical guide and assessment on using ChatGPT to conduct grounded theory: Tutorial. *Journal of Medical Internet Research*, 27:e70122, 2025. doi: [10.2196/70122](https://doi.org/10.2196/70122) 1
- [67] Y. Zhao, J. Wang, L. Xiang, X. Zhang, Z. Guo, C. Turkay et al. LightVA: Lightweight visual analytics with LLM agent-based task planning and execution. *IEEE Transactions on Visualization and Computer Graphics*, 31(09):6162–6177, 2025. doi: [10.1109/TVCG.2024.3496112](https://doi.org/10.1109/TVCG.2024.3496112) 2
- [68] M. Zhong, P. Wang, and A. Field. HICode: Hierarchical inductive coding with LLMs. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pp. 31060–31078, 2025. doi: [10.18653/v1/2025.emnlp-main.1580](https://doi.org/10.18653/v1/2025.emnlp-main.1580) 3
- [69] Y. Zhou, Y. Yuan, K. Huang, and X. Hu. Can ChatGPT perform a grounded theory approach to do risk analysis? an empirical study. *Journal of Management Information Systems*, 41(4):982–1015, 2024. doi: [10.1080/07421222.2024.2415772](https://doi.org/10.1080/07421222.2024.2415772) 1